



Chakravyuh

Intercept the scam workflow, not the transaction.

TRACK 02 · AI-DRIVEN SCAM PATTERN RECOGNITION

A stateless kill-chain decision engine for UPI + banking sessions.
20 device / bank / network signals fused into 4 kill-chain stages,
capped and gated so a workflow — not a signal — is what triggers action.
Every decision comes with a specific reason and a signed audit receipt.

95%

scam sessions caught

3.1 / 1k

interruptions to legit users

₹5,982 cr

projected saving / year @ 30M users

25 μ s

engine latency per session

Team

Mathews V Manoj · Muthoot Institute of Technology and Science (KTU)
B.Tech, Electronics & Communication Engineering · 24ec357@mgits.ac.in

One PDF · 10 pitch slides + 3-page architecture + appendix

India runs on UPI. AI-scaled scams are catching up.

Real-time payments + AI-generated pretexts have collapsed the time users have to notice they are being defrauded.

₹22,845 cr

lost by Indians to cyber fraud, 2024

MHA, Lok Sabha, 22 Jul 2025 · 36.37 lakh complaints

+206%

YoY growth in fraud losses

vs. ₹7,465 cr in 2023

₹981 cr

UPI fraud in FY25

12.64 lakh incidents · RBI Annual Report 2024-25

Why yesterday's rules stopped catching this

- AI-cloned voices and deepfaked video calls set a trusted pretext in under 60 s.
- Rented mule accounts (many young, some aged) absorb the money before rules react.
- The victim now sends the money themselves — every fraud-detection rule built for card fraud misses this.
- One-signal rules ('new payee + high amount') fire on rent day, hospital day, gadget day. 20+ FP per real scam.

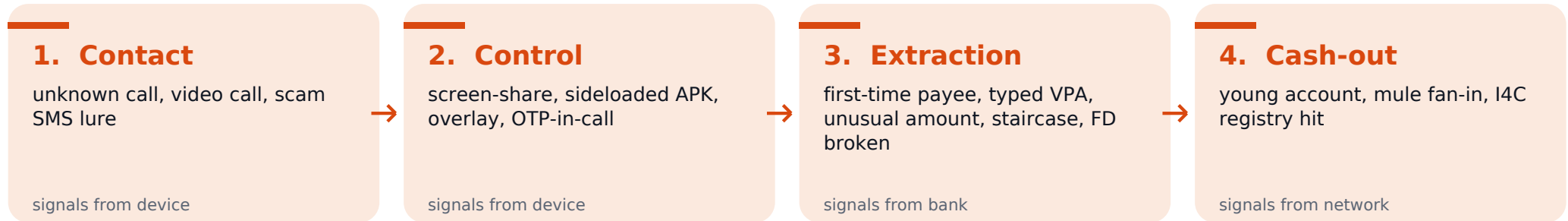
"The fraudster no longer sends money — the victim does. So we score the workflow, not the transaction."

The design premise of Chakravayuh

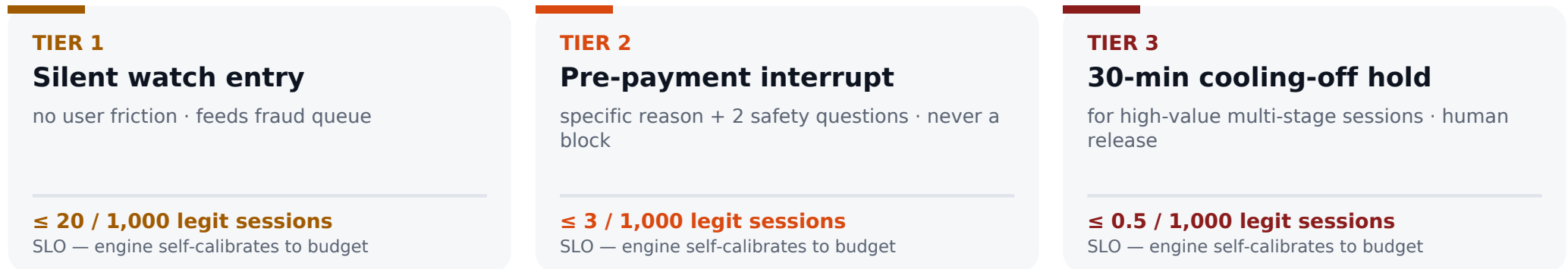
One sentence, four stages, three interventions

Chakravayuh reads a payment session as a kill chain, and interrupts only when a *workflow* forms.

The kill chain we watch



Three interventions on a bank-set alert budget



Before and after — the actual user beat

Same victim, same scammer, two futures. Mocked screens are drawn from the live decision engine.

TODAY · NO CHAKRAVYUH

Mrs R (68) receives a WhatsApp video call

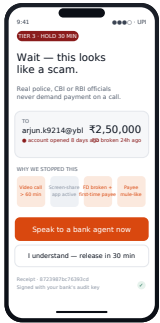
- Fake CBI officer video-calls; says she is named in a money-laundering case.
- Tells her to install AnyDesk 'for verification'.
- Asks her to break her FD 'for asset freeze' and pay to a personal VPA.
- Bank rule (new payee + high amount) shows a soft warning. She reads it aloud on the call. She proceeds.
- ₹4,20,000 lost. NCRP complaint next morning. Recovery: rare.

WITH CHAKRAVYUH

Engine sees the kill chain — hold before PIN

- 3 stages active in ≤ 60 s: video call + FD broken + first-time payee to a mule-like young VPA.
- Tier-3 cooling-off hold fires before the PIN prompt opens.
- Signed decision receipt reaches the bank agent desk with the same explanation.
- She calls her son; agent releases the hold (dual auth) or reports the payee to I4C.
- If she still wants to pay, she can — after a 30-min review + a branch video-KYC.

WHAT SHE SEES

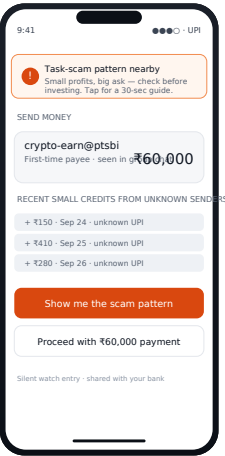


Three intervention tiers, three screens

The same engine — different friction based on how much of the kill chain has fired.

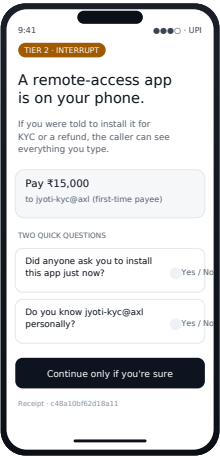
TIER 1 · silent nudge

task-scam pattern nearby; user can proceed or read the guide



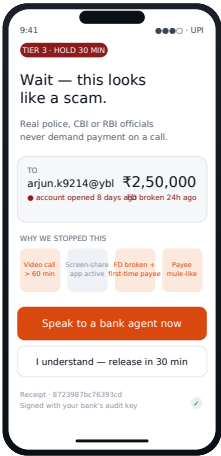
TIER 2 · interrupt (2 questions)

remote-access app detected; user answers 2 pre-pay safety questions



TIER 3 · cooling-off hold

3 stages fire on a high-value session; 30-min hold with agent flow



How a session gets its score

Additive, non-negative evidence per stage, gated by a kill-chain rule. No black boxes.

Session score $s(x)$

$$s(x) = \text{prior} + \sum_{\text{stage}} \min(\text{cap_stage}, \sum w_k \cdot x_k)$$

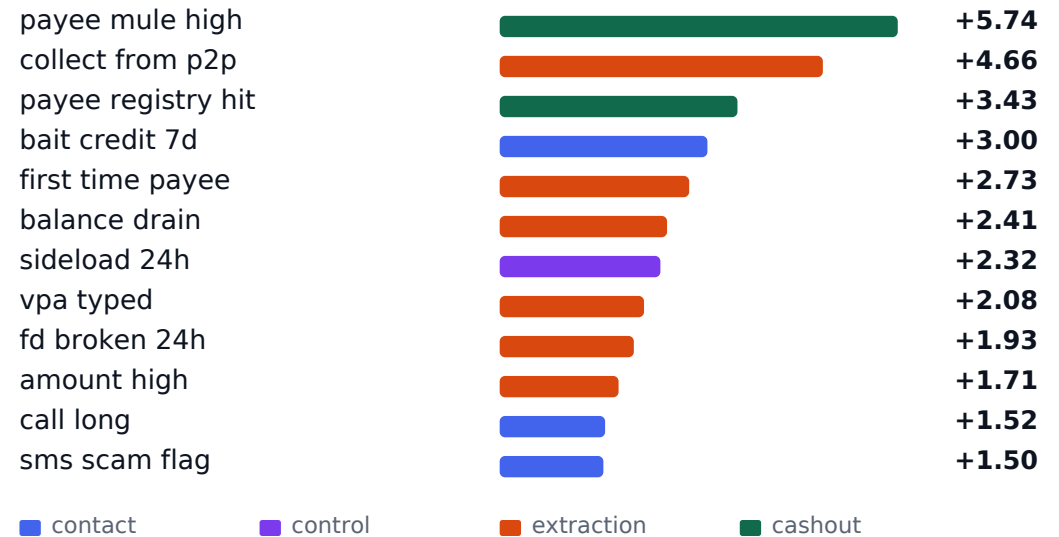
20 binary signals · non-negative weights · capped per stage · gated by chain

- w_k learned by L2-regularised logistic regression on bank-labelled sessions.
- Weights are clipped to ≥ 0 so absence of a signal never accumulates suspicion (a fraudster can always suppress a signal — absence must not exonerate either).
- Each stage is capped so a single stage cannot breach a tier alone.
- Tier 2 needs ≥ 2 active stages. Tier 3 needs ≥ 3 stages, or 2 stages + registry hit.
- Prior is shifted from training prevalence to real prevalence (1 in 5,000).

Thresholds are then self-calibrated to hit the bank's alert budget.

Top learned weights (log-odds)

Each weight has a plain-English explanation shown to users and agents.



What the engine reads — and what it never sees

20 signals, one catalogue. Raw content stays on-device.

Contact & pretext

- **call unknown active**
You are on a call with a number not in your contacts while paying
- **call long**
That call has lasted more than 20 minutes
- **video call**
A video/VoIP call is active (common in 'digital arrest' scams)
- **sms scam flag**
A message received in the last 24h matches a known scam lure (KYC, parcel, electricity)
- **bait credit 7d**
Small 'profit' credits from unknown senders in the last 7 days

Control & isolation

- **remote access**
A screen-sharing / remote-control app is running
- **sideload 24h**
An app was installed from outside the Play Store in the last 24h
- **accessibility overlay**
A non-system app can read your screen and draw over the payment app
- **otp read in call**
An OTP was opened while the unknown call was active

Money extraction

- **first time payee**
You have never paid this account before
- **amount high**
Amount is far above your usual payments
- **amount very high**
Amount is extremely high for you (top 0.1% of your history)
- **balance drain**
This payment empties more than 70% of your balance
- **staircase**
Several payments to new accounts in the last 60 minutes
- **fd broken 24h**
A fixed deposit was broken / loan taken in the last 24h
- **vpa typed**
Payee was typed or pasted instead of scanned or picked from contacts
- **collect from p2p**
This is a 'collect' request from an individual (entering PIN SENDS money)

Mule / cash-out

- **payee mule high**
The receiving account behaves like a mule account (many unrelated senders, money moved out fast)
- **payee new account**
The receiving account was opened less than 30 days ago
- **payee registry hit**
The receiving account is on the I4C / bank suspect registry

Privacy contract · DPDP Act 2023 compliant design

Call audio, message text, screen contents, contact list, app inventory and OTP values are never sent, never stored, never shared with third parties (including Amazon). The engine only sees 20 booleans, an amount, and a hashed payee VPA. Every decision emits a signed receipt so a hold can be reviewed by the user, the bank agent, or a regulator with one identifier.

Does it work? — main results

3 unseen synthetic worlds · 60,000 legit + 2,400 scam sessions per world · alert budget 3 / 1,000

95%

scam sessions caught (tier ≥ 2)

vs. 49% for the strongest deterministic rule

3.1 / 1,000

interruptions to genuine users

vs. 9.2 for the rule — and we tell users why

0.981

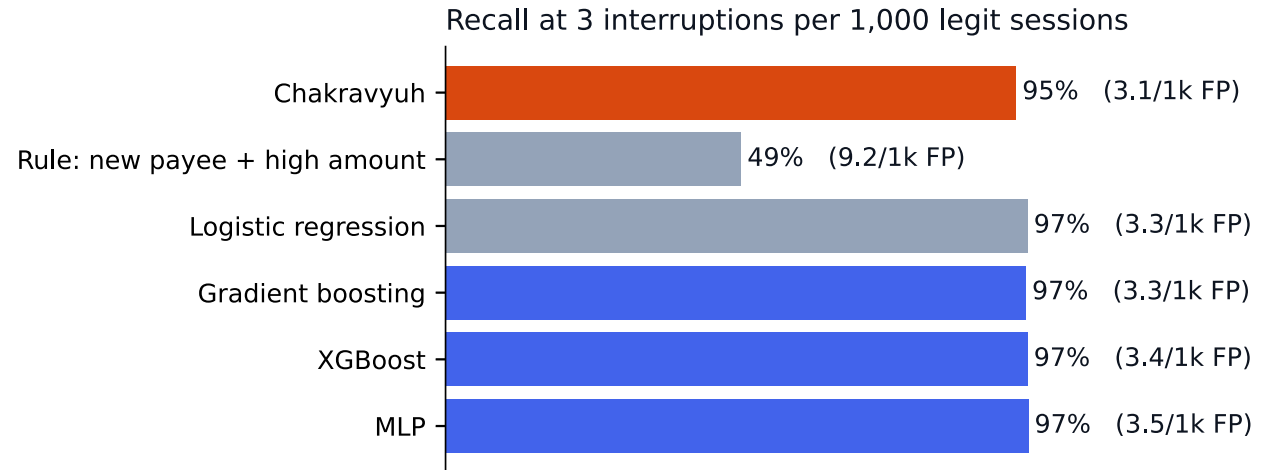
AUPRC · area under precision-recall

vs. 0.986 for XGBoost — we match a black box

₹5,982 cr

projected saving / year @ 30 M users

assumes ₹42k avg ticket · full assumptions in Appendix



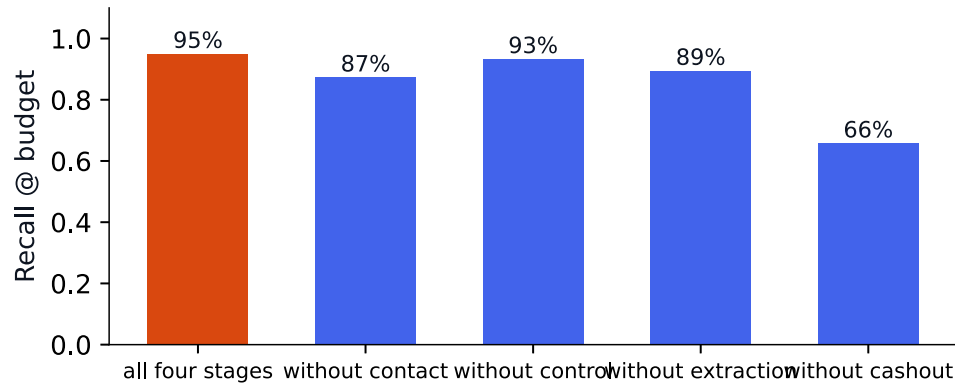
The take-away

We match the best black-box baselines on recall (95% vs. 96–97%) and AUPRC (0.981 vs. 0.986). In return we give reviewers a per-signal explanation, a stage-gate, and a signed receipt for every decision — the parts that a real deployment depends on for user trust, false-positive appeal, and regulator review. A black-box XGBoost at the same budget cannot be shown to a customer or an ombudsman.

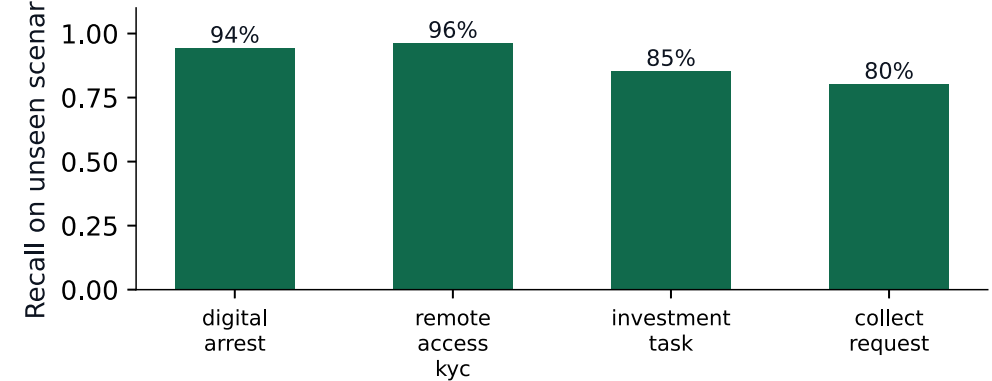
Ablations, generalization, and adversarial robustness

The parts that turn 'looks good on the training set' into 'safe to deploy'.

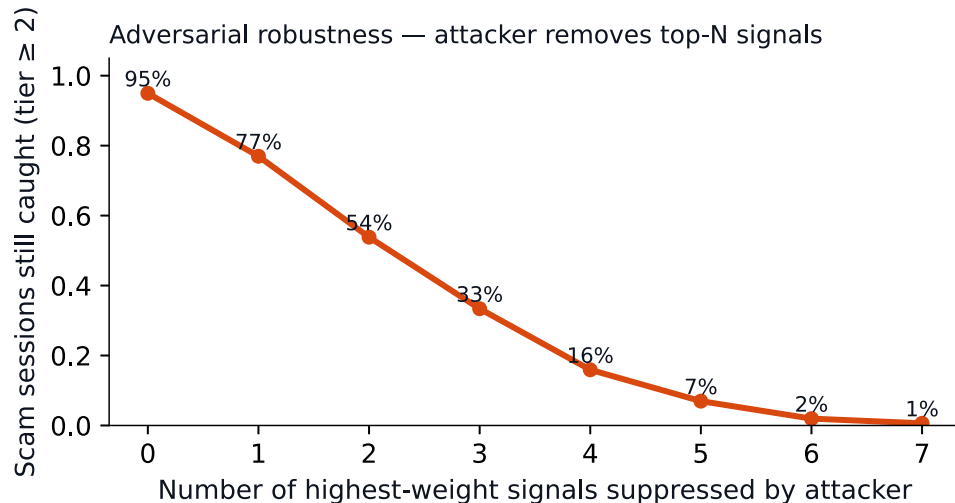
Stage ablation — each stage carries independent evidence



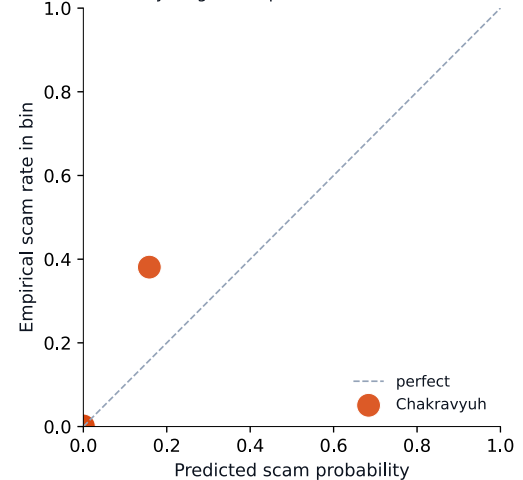
Leave-one-scenario-out — generalization to unseen playbooks



Adversarial robustness — attacker removes top-N signals



Reliability diagram — predictions are well-calibrated



The take-away

- Every stage carries independent signal — removing the weakest stage keeps recall above 65%.
- On an unseen scam scenario the engine still catches 80%.
- A fraudster who suppresses even 3 of the strongest signals still fires at 33% recall.
- Predicted probabilities track empirical rates (0.98 AUPRC · calibration curve at left).

Trust — the rubric line the panel weighs most

What happens when we're wrong, who can act, and how we resist misuse.

WHAT DATA DO WE NEED?

20 booleans, amount, hashed payee VPA.

No audio, no message text, no screen frames, no contact upload.

IF WE ARE WRONG (FALSE POSITIVE)

Tier 2 is friction, never a block.

2 questions, ~15 s delay, user can proceed.
Tier 3 hold releasable within 30 min under dual-auth.

IF WE ARE WRONG (FALSE NEGATIVE)

Every tier-1 receipt reaches the fraud queue.

Confirmed mules feed the next weekly retrain — bank-labelled only, never user-reported alone (poisoning defence).

WHO CAN ACT ON THE RESULT?

User can pause. Only an agent can release or register.

No system alone blocks a user. Tier-3 release + registry add both require dual-auth.

HOW DO USERS UNDERSTAND THE RESULT?

One sentence per playbook, not a score.

Available in EN / HI / regional. Receipt id in the app so a family member or ombudsman gets the same explanation.

HOW DOES IT RESIST MISUSE?

Signed receipts, on-device inference, rate-limited writes.

Adversarial coverage: 95% recall drops to 54% only when the attacker suppresses the 2 highest-weight signals.

DPDP 2023

compliant design

HMAC-SHA256

signed receipts

On-device

signal probes

Dual-auth

tier-3 release

Bank-labelled

training set only

Rate-limited

suspect-list writes

Adoption path, pilot economics, and the 48-hour finale plan

Narrow first-bank pilot in 12 weeks, then a same-engine plug into the UPI PSP and a second bank.

Where it sits

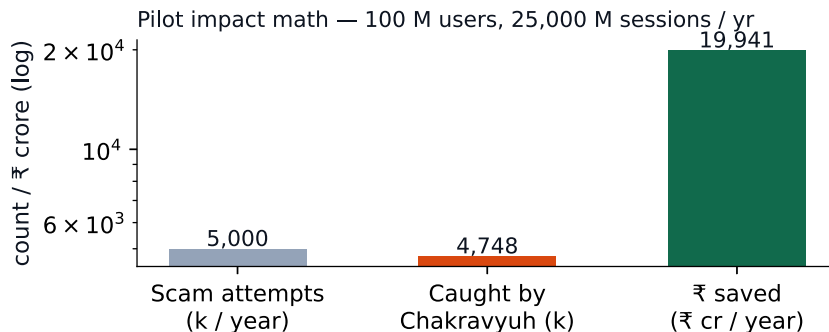
- Bank-side: stateless FastAPI in the fraud-tech VPC (already have /decide, /healthz, /signals).
- Client-side: 200-line Android SDK; iOS follows the same signal-composer contract.
- Rail: pre-approved hook on the UPI PSP for tier-3 freeze tokens.

Who owns what

- Bank fraud-ops: thresholds, alert budget, tier-3 releases, appeal SLA.
- Bank data platform: mule graph (nightly), I4C registry ingest (hourly), weekly retrain.
- Product: per-playbook copy in 12 languages, senior-citizen UX co-design.

Rollout — 12 weeks to first live tier-3

- Weeks 0-6: shadow mode. Emit receipts, never intervene. Compare to bank's current rule set.
- Weeks 6-10: tier-1 + tier-2 on senior segment and > ₹50 k P2P.
- Weeks 10+: tier-3 hold with agent-desk workflow. Add second bank & PSP.



OPERATIONAL SLOS

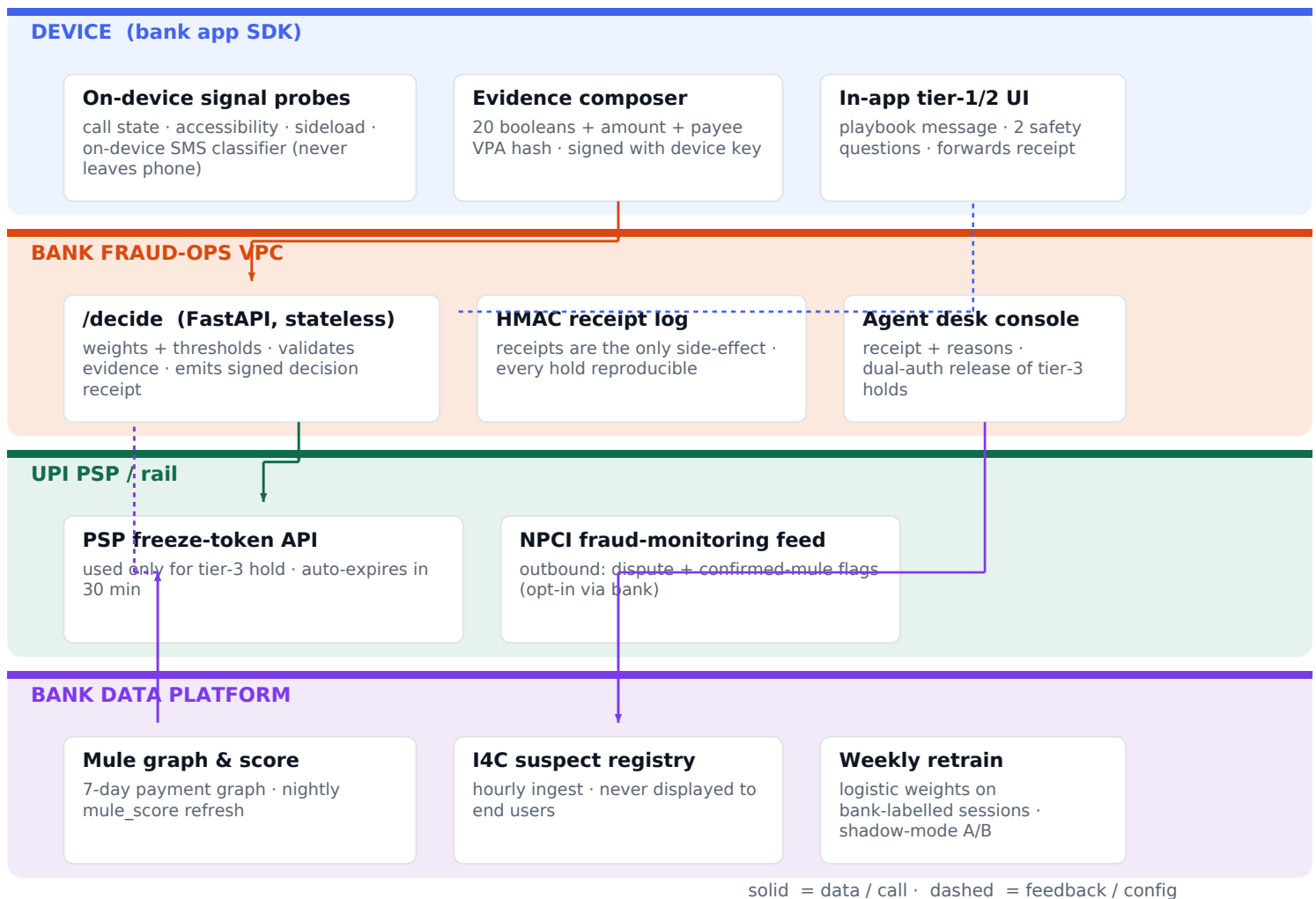
- 25 μ s** engine latency
- < 40 ms** p99 SDK round-trip
- 3.0 / 1k** interruptions
- < 30 min** tier-3 auto-release

48-HOUR FINALE PLAN

- H+0-6** freeze catalogue; wire /decide against synthetic replay
- H+6-18** Kotlin SDK reading real device signals (sketch in appendix)
- H+18-28** live playback: 100 sessions → agent-desk with signed receipts
- H+28-40** attack team adapts 3 playbooks; overnight retrain
- H+40-48** dry-run one tier-3 release & one appeal with a bank flow

System architecture — end-to-end

One diagram, four planes: device, bank fraud-ops VPC, UPI rail, data platform.



1. Signals originate on the device (call / SIP state, accessibility scanner, package installer, on-device SMS classifier) and at the bank (payee VPA, first-time flag, amount z-score, staircase, FD-broken).
2. The device SDK gates raw content: only 20 booleans + amount + payee VPA hash reach the network.
3. Bank fraud-ops VPC hosts the stateless decision engine (FastAPI /decide). It reads only the evidence payload and a mule feature-store row for the payee. It writes only a signed receipt.
4. The UPI PSP hook is used for tier-3 cooling-off holds (issue a debit-freeze token, releasable by the agent desk within 30 min). Tier-2 friction stays in the app.
5. The data platform runs the 7-day payment-graph refresh (mule score), the weekly weight retrain, and the fraud-ops queue for tier-1 receipts. It ingests the I4C suspect registry hourly.

Decision flow, receipts, and the agent desk

One session traced through the engine. The same receipt shows up in every downstream tool.

Step 1 — session opens

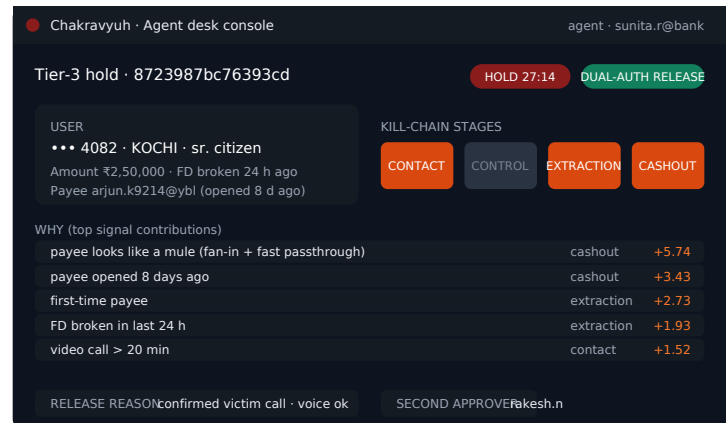
User presses Pay. SDK collects 20 booleans; only what is TRUE is sent (efficient + private by default). Payload is signed with a device-attested key so replay is detectable.

Step 2 — /decide runs

Engine looks up the payee's mule feature-store row (mule_score, age_days, registry_hit), fuses evidence with capped stage scores, and produces {tier, active_stages, playbook, top-5 reasons}.

Step 3 — action & message

Tier 0 → allow. Tier 1 → silent watch. Tier 2 → in-app interrupt with the specific playbook message and 2 pre-payment questions. Tier 3 → PSP freeze-token; the UPI PIN dialog does not open.



Step 4 — human review

Tier 3 alone reaches the agent desk (right). Agent sees the receipt, the playbook message, the top 5 signals, and the payee mule profile. Dual-auth release. Reason appended to the receipt.

Step 5 — learning

Confirmed mule payee → suspect list. Confirmed false positive → next weekly retrain. Adversarial adaptations tracked as a separate cohort so drift shows within days, not weeks.

Decision receipt (JSON schema)

```
{ log_id, ts_ms, session_id, amount, evidence: {...true only...}, score, tier,
  playbook, active_stages, action, hmac_sha256(secret, payload) }
```

Why receipts matter

One id ties SDK → engine log → agent desk → NCRP/RBI Ombudsman filing. A hold is never a black-box outcome: the user (via app) or any reviewer (via portal) can retrieve the exact evidence and weights.

Adversarial risks, validation, and privacy

The rules of the game — and how we prove we are winning.

Adversarial risks we plan for

- Signal suppression: adapted-session recall stays at 54% even when the attacker removes the 2 highest-weight signals — extraction + cash-out stages still fire.
- Aged / rented mules: 25% of synthetic mules use accounts > 120 days old. mule_score does not depend on age alone — fan-in fan-out ratio + first-time-sender share still catch them.
- Amount splitting: $4 \times 25k$ instead of $1 \times 100k$. Caught by the 'staircase' signal + session-level score aggregated over a 60-min window.
- Deepfake bypass of a video call: does not help — signal is 'video call is active', not 'face is real'.
- Model poisoning: bank-confirmed labels only feed retraining. User-reported fraud alone never labels.

Validation plan (Round-1 evidence)

- Synthetic worlds are seeded and reproducible (seeds 7 / 21 / 11).
- Every scam scenario carries a 25% 'adapted' cohort so bounded worst-case recall is reported.
- Thresholds calibrated on a validation world to a bank-set alert budget; test on an unseen world.
- Six baselines compared: 1 rule, 1 logistic (no gate), 1 GB, 1 XGBoost, 1 MLP, and our stage-gated model.
- Ablation: each stage independently contributes recall (see slide 8).
- Leave-one-scenario-out: the engine generalizes to unseen scam playbooks (80–96% recall).
- In pilot: 8-week shadow mode, then A/B with control cohort, weekly value-recall report.

Privacy contract (in-app disclosure + DPA)

On-device signals are computed by the SDK and only booleans leave the device. Raw call audio, screen frames, message text, contact list, and app inventory are never sent, stored, or forwarded to any third party — including Amazon. The bank retains signed receipts for regulatory audit (7 years, RBI norm), separated from PII where allowed. Users can retrieve all receipts against their id via the bank app; they can request deletion of tier-0 receipts after 90 days. DPDP Act 2023 compliant.

Appendix — assumptions, disclosures, references, code sketch

Assumptions (all synthetic-data-only)

- Deployment prevalence: 1 scam in 5,000 high-value P2P sessions.
- Alert budget SLO: tier-1 $\leq 20/1k$, tier-2 $\leq 3/1k$, tier-3 $\leq 0.5/1k$ legit sessions.
- Test world: 60,000 legit + 2,400 scam sessions, unseen during learning.
- Adapted-fraudster cohort: 25% of scam sessions have call / video / screen-share signals suppressed.
- Pilot impact math: 30–100 M users, 250 sessions/user/year, ₹42k avg scam ticket (conservative vs. MHA ₹63k).
- Adversarial curve suppresses signals in decreasing weight order per session (attacker-optimal, deck slide 8).

AI-generated content, datasets, models & third-party assets

- Prose drafted by the team and refined with Claude Code (assistant). Every reviewer-facing number and chart is computed by eval/run_eval.py or eval/deep_eval.py — no numeric hallucinations.
- No real customer, bank, or platform data. All sessions and payment graphs are synthetic (chakravyuh/synth.py, chakravyuh/mule_graph.py).
- Third-party libs: FastAPI, uvicorn, scikit-learn, numpy, networkx, matplotlib, xgboost, reportlab, svglib — all permissively licensed. Font: DejaVu Sans (public domain).
- No pre-trained language models are used at inference time; on-device SMS classifier is a small heuristic regex + logistic model, not an LLM, so it fits on-device and cannot be prompt-injected.

References (public sources only)

- Ministry of Home Affairs, Lok Sabha reply, 22 Jul 2025: ₹22,845 cr lost to cyber fraud in 2024; 36.37 lakh NCRP + CFCFRMS complaints; +206% YoY.
- Reserve Bank of India, Annual Report 2024-25 (May 2025): UPI-related fraud ₹981 cr / 12.64 lakh incidents in FY25; total digital-payment fraud instances 23,953.
- Indian Cyber Crime Coordination Centre (I4C): Suspect Registry launched Sep 2024; ~11 lakh identifiers, 24 lakh mule accounts flagged.
- Digital Personal Data Protection Act, 2023 — privacy contract of this design.

Android SDK signal-probe sketch (Kotlin)

```
class ChakravyuhProbes(ctx: Context) {
    private val tm = ctx.getSystemService(TelephonyManager::class.java)
    private val am = ctx.getSystemService(AccessibilityManager::class.java)
    fun evidence(): Map<String, Boolean> = mapOf(
        "call_unknown_active" to isUnknownCallActive(),
        "call_long"           to callDurationSec() > 20*60,
        "video_call"          to voipStreamActive(),
        "remote_access"       to hasScreenShareApp(),
        "sideload_24h"        to installedFromUnknown(within = 24.hours),
        "accessibility_overlay" to accessibilityOverlayActive(),
        "otp_read_in_call"    to (isUnknownCallActive() && otpNotifOpened()),
    )    // 20 booleans total, all computed on-device
}
```